SELECTED WORK

Code Samples

A curated showcase of original engineering work.

Aaron K. Clark

Six original projects · seven languages

Rust · Go · Python · JavaScript · SQL · C# · GDScript

PROJECT

Scylla

A reverse-engineering platform with a durable core and disposable heads.

> A hexagonal RE platform: a durable reverse-engineering domain core surrounded by swappable protocol “heads” (MCP first). The core never depends on a head, so the interface can be replaced without touching the analysis logic. The sample is the submit→poll job handle that runs a long analysis on a background task — a 200 MB firmware analysis can’t block the caller — with truthful status and exactly-once result delivery.

Rust | Async / Tokio | Hexagonal architecture | Architect & author

An async submit→poll job handle: type-safe lifecycle, atomic IDs, and one-shot result semantics for background analysis.

```rust
//! DD-019 — a submit→poll **job handle** for the long-running `analyze` (materialize).
//!
//! DD-019's decision: synchronous request/response everywhere, EXCEPT a job handle
//! (submit → poll) for the one call that can't be a blocking wait — analyzing a binary (a
//! 200 MB firmware analysis can't block the caller). Streaming + fine-grained cancellation
//! are deliberately deferred. The async producer work lives here on the engine side; the
//! client port (`scylla-port`) stays a pure, synchronous model consumer (DD-009) — a head
//! submits the analyze, polls the handle, and `Session::open`s the `Program` when it lands.

use std::future::Future;
use std::sync::atomic::{AtomicU64, Ordering};
use std::sync::{Arc, Mutex};

use scylla_model::Program;
use tokio::task::JoinHandle;

static NEXT_JOB_ID: AtomicU64 = AtomicU64::new(1);

/// An opaque analyze-job identifier (process-local, monotonic).
#[derive(Clone, Copy, Debug, PartialEq, Eq, Hash)]
pub struct JobId(pub u64);

/// Lifecycle state of a submitted analyze job — what [`AnalyzeJob::status`] returns.
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub enum JobStatus {
    /// Still analyzing.
    Running,
    /// Finished; a `Program` is ready to [`take`](AnalyzeJob::take).
    Succeeded,
    /// Finished with an error (take the result for the message).
    Failed,
}

/// The slot the detached task writes its outcome into. `ok` is retained so `status()` stays
/// truthful even after the (one-shot) result has been taken.
enum Slot {
    Running,
    Done {
        ok: bool,
        result: Option<Result<Program, String>>,
    },
}

/// A handle to a long-running analyze submitted via [`AnalyzeJob::submit`] / [`submit_analyze`].
/// The work runs on a detached Tokio task; a consumer polls [`status`](Self::status) and
/// [`take`](Self::take)s the result once it lands, or [`join`](Self::join)s to await it.
pub struct AnalyzeJob {
    id: JobId,
    slot: Arc<Mutex<Slot>>,
```

An async submit→poll job handle: type-safe lifecycle, atomic IDs, and one-shot result semantics for background analysis.

```
    task: JoinHandle<()>,
}

impl AnalyzeJob {
    /// Submit a future producing a `Program` as a background analyze job. The error is
    /// stringified at the boundary so the outcome is self-contained. Requires a Tokio runtime.
    pub fn submit<F, E>(fut: F) -> Self
    where
        F: Future<Output = Result<Program, E>> + Send + 'static,
        E: std::fmt::Display + Send + 'static,
    {
        let id = JobId(NEXT_JOB_ID.fetch_add(1, Ordering::Relaxed));
        let slot = Arc::new(Mutex::new(Slot::Running));
        let writer = Arc::clone(&slot);
        let task = tokio::spawn(async move {
            let outcome = fut.await.map_err(|e| e.to_string());
            let ok = outcome.is_ok();
            *writer.lock().unwrap() = Slot::Done {
                ok,
                result: Some(outcome),
            };
        });
        AnalyzeJob { id, slot, task }
    }

    /// This job's id.
    pub fn id(&self) -> JobId {
        self.id
    }

    /// Poll the job's lifecycle state without consuming it or its result.
    pub fn status(&self) -> JobStatus {
        match &*self.slot.lock().unwrap() {
            Slot::Running => JobStatus::Running,
            Slot::Done { ok: true, .. } => JobStatus::Succeeded,
            Slot::Done { ok: false, .. } => JobStatus::Failed,
        }
    }

    /// Take the finished result, or `None` while still running. Yields the outcome exactly
    /// once; a later call after a successful take returns `None` (use [`status`](Self::status)
    /// to re-read success/failure).
    pub fn take(&self) -> Option<Result<Program, String>> {
        match &mut *self.slot.lock().unwrap() {
            Slot::Done { result, .. } => result.take(),
            Slot::Running => None,
        }
    }
```

PROJECT

AdmiralBBS

A security-hardened, clean-room reimplementation of a 1990s ANSI BBS.

> A from-scratch BBS server written in memory-safe Go. The memory-safe language removes the buffer-overflow bug class; this layer removes the other one that matters for a terminal service — control-character and ANSI-escape injection. The sample is the input security boundary: a pure, fuzzable function that length-bounds and sanitizes everything a caller sends before it can reach a parser or be echoed to a terminal.

Go | Security hardening | Clean-room

A pure, fuzz-testable sanitizer that strips control bytes and ANSI/CSI escape sequences — the service's injection boundary.

```go
package session

// Hardened input handling. This is the security boundary named in
// DECISIONS.md: caller input is length-bounded and sanitised before it reaches
// any parser or is echoed back to a terminal. A memory-safe language already
// removes the buffer-overflow class; this layer removes the
// control-character / escape-sequence injection class ("packet injection").

// MaxLineLen bounds a single line read so a hostile caller cannot make the
// server allocate without limit.
const MaxLineLen = 4096

// allowedControl is the small set of control bytes we let through unchanged.
func allowedControl(b byte) bool {
    switch b {
    case '\t', '\n', '\r':
        return true
    default:
        return false
    }
}

// SanitizeInput strips dangerous bytes from a caller-supplied buffer:
//   - control characters (0x00–0x1F, 0x7F) except TAB/CR/LF are dropped;
//   - ANSI/VT escape sequences are removed entirely — an ESC followed by '['
//     (CSI) is consumed through its final byte (0x40–0x7E); an ESC followed by
//     any other byte drops both (the ANSI-BBS "legal function" range);
//   - printable bytes (including high CP437 bytes 0x80–0xFF) pass through.
//
```

A pure, fuzz-testable sanitizer that strips control bytes and ANSI/CSI escape sequences — the service's injection boundary.

```
// It is a pure function so it can be unit-tested and fuzzed directly.
func SanitizeInput(in []byte) []byte {
    out := make([]byte, 0, len(in))
    for i := 0; i < len(in); i++ {
        b := in[i]
        switch {
        case b == 0x1B: // ESC — start of an escape sequence
            if i+1 < len(in) && in[i+1] == '[' {
                // CSI: skip until a final byte in 0x40–0x7E.
                i += 2
                for i < len(in) && (in[i] < 0x40 || in[i] > 0x7E) {
                    i++
                }
                // i now points at the final byte (or end); loop's i++ consumes it.
            } else {
                // ESC + single function char: drop both.
                i++
            }
        case b < 0x20 || b == 0x7F:
            if allowedControl(b) {
                out = append(out, b)
            }
            // else: drop the control byte.
        default:
            out = append(out, b) // printable ASCII or high CP437 byte
        }
    }
    return out
}
```

PROJECT

TimeTrackerAPI

An open-source Node.js + PostgreSQL rewrite of a commercial time-tracking API.

> A multi-tenant REST API with API-key authentication. Two samples from one project, in two languages: the auth middleware that hashes the incoming key before lookup and scopes every request to its company, and the hand-authored Postgres schema — documented conventions, soft-delete, and partial indexes tuned to the hot read paths.

JavaScript / Node | PostgreSQL / SQL | REST | Multi-tenant

app/middleware/auth.js part 1/2

API-key auth: SHA-256 hashed-key lookup through the model layer, with company-scoped access control and fail-closed errors.

```
/**
 * Hash an authKey for lookup. Migration 20260518000000 converted
 * ApiKey.akKEY / ApiMaster.amKEY columns from UUID to TEXT and
 * replaced row values with SHA-256 hex digests. Operator tokens
 * issued before the migration keep working because the API hashes
 * the incoming header here before the SQL lookup.
 *
 * SHA-256 unsalted (vs bcrypt/argon2id) because API tokens are
 * high-entropy (UUID v4 = 122 bits); brute force against a leaked
 * hash table is impractical. Hashing is to prevent direct replay
 * if the DB leaks, not to protect a low-entropy password.
 */
function hashKey(rawKey) {
    return crypto.createHash('sha256').update(String(rawKey)).digest('hex');
}

/**
 * Why model calls instead of raw `sequelize.query`:
 *
 * P5-M reworked this module to route every DB hit through the
 * Sequelize model layer (`ApiMaster.findOne`, `Customer.findByPk`,
 * etc.) for testability. `vi.mock('../config/db.config.js', ...)`
 * intercepts the module export; the raw `db.sequelize.query` path
 * had a nested CJS-require interplay that often slipped past the
 * mock, leaving tests exercising the real (unreachable in tests) DB.
 * Going through the models means every code path here is testable
 * with the same model-stub fixtures the rest of the api tests use.
 *
 * Performance is no worse — these are single-row primary-key
 * lookups; Sequelize's per-row instantiation overhead is in the
 * sub-millisecond range and dwarfed by the network round-trip.
 *
 * The archive filter (`<arch>: false`) is no longer hand-rolled in
```

app/middleware/auth.js part 2/2

API-key auth: SHA-256 hashed-key lookup through the model layer, with company-scoped access control and fail-closed errors.

```
 * the WHERE clause. P2-E added `defaultScope` to every model with
 * an archive column, so the soft-delete filter is implicit.
 */

async function isMaster(authKey) {
    if (!authKey || authKey.length === 0) return false;
    try {
        const row = await getDb().ApiMaster.findOne({
            where: { amKEY: hashKey(authKey) },
            attributes: ['amId'],
        });
        return !!(row && typeof row.amId === 'number' && row.amId > 0);
    } catch (error) {
        log.error({ err: error }, 'auth.isMaster query failed');
        return false;
    }
}

async function getCompanyId(authKey) {
    if (!authKey || authKey.length === 0) return -1;
    try {
        const row = await getDb().ApiKey.findOne({
            where: { akKEY: hashKey(authKey) },
            attributes: ['akCompanyId'],
        });
        if (!row) return -1;
        const cid = row.akCompanyId;
        return typeof cid === 'number' && cid > 0 ? cid : -1;
    } catch (error) {
        log.error({ err: error }, 'auth.getCompanyId query failed');
        return -1;
    }
```

setup/TimeEntry.sql sql

A documented multi-tenant table: schema conventions, soft-delete columns, and partial indexes matched to the listing queries.

```sql
--
-- TimeEntry table for the /v1/timeentry endpoints.
-- Apply after the base TimeTracker.sql with:
--   sudo -u postgres psql -d timetracker -f setup/TimeEntry.sql
--
-- The table follows the existing schema conventions:
--   - dbo schema
--   - 2-3 char column prefix (te = TimeEntry)
--   - SERIAL primary key
--   - soft-delete via teArch / teArchiveDate (matches Customer.custArch
--     and ApiKey.akArchive patterns)
--   - timestamps stored as TIMESTAMPTZ

SET search_path TO dbo;

CREATE TABLE IF NOT EXISTS dbo."TimeEntry" (
    "teId"          SERIAL          PRIMARY KEY,
    "teCustId"      INTEGER         NOT NULL REFERENCES dbo."Customer"("custId"),
    "teCompId"      INTEGER         NOT NULL,
    "teDescription" TEXT,
    "teStartedAt"   TIMESTAMPTZ     NOT NULL,
    "teEndedAt"     TIMESTAMPTZ,
    "teMinutes"     INTEGER,
    "teBillable"    BOOLEAN         NOT NULL DEFAULT TRUE,
    "teArch"        BOOLEAN         NOT NULL DEFAULT FALSE,
    "teArchiveDate" TIMESTAMP(3) WITHOUT TIME ZONE
);

ALTER TABLE dbo."TimeEntry" OWNER TO timetracker;

-- Hot-path indexes:
--  * listByCompany: filtered by (teCompId, teArch) with optional
--    teCustId and date-range on teStartedAt.
--  * getById / update / remove: PK lookup is already free.
CREATE INDEX IF NOT EXISTS "TimeEntry_company_started_idx"
    ON dbo."TimeEntry" ("teCompId", "teArch", "teStartedAt" DESC);

CREATE INDEX IF NOT EXISTS "TimeEntry_customer_started_idx"
    ON dbo."TimeEntry" ("teCustId", "teStartedAt" DESC)
    WHERE "teArch" = FALSE;
```

PROJECT

OSApplyTrack

An open-source job-search application tracker, running in production.

> An ASP.NET / .NET API for tracking job applications and generating materials. The sample is the cover-letter drafter: it builds a strict, structured LLM prompt from the tenant's own résumé (not hardcoded facts), forbids invented claims, and validates the model's output before it is ever persisted — dependency-injected and unit-testable.

C# | ASP.NET / .NET | LLM integration

An injectable service that builds an anti-hallucination LLM prompt from structured data and validates output before saving.

```
using ApplyTrack.Api.Data;
using ApplyTrack.Api.Llm;

namespace ApplyTrack.Api.Materials;

/// <summary>
/// Drafts a tailored cover letter for one application — the multi-tenant, server-side
/// heir to the original <c>materials.py</c>. The prompt's anti-hallucination brief is
/// the tenant's own structured <see cref="Resume"/> (not hardcoded facts), and the
/// output is plain text/Markdown (the LaTeX/PDF path is a later module).
/// </summary>
public sealed class CoverLetterDrafter
{
    private readonly ILlmClient _llm;

    public CoverLetterDrafter(ILlmClient llm) => _llm = llm;

    // Which strength the letter leads with — mirrors the original lane switch.
    private static readonly Dictionary<string, string> LaneLead = new()
    {
        ["dotnet"] = ".NET / backend-engineering",
        ["devrel"] = "developer-relations / developer-enablement",
        ["ai"] = "AI-agent / applied-AI",
    };

    public async Task<string> DraftAsync(
        AppFields app, Resume resume, EffectiveLlmConfig cfg, CancellationToken ct = default)
    {
        if (resume.IsEmpty)
            throw new AppValidationException("add your résumé in Résumé settings before drafting a cover letter");

        var (system, user) = BuildPrompt(app, resume);
        var body = (await _llm.CompleteAsync(system, user, cfg, ct)).Trim();

        // Reject empty/implausible output rather than save a broken letter.
        if (body.Length is < 40 or > 6000)
            throw new LlmUnavailableException("the model returned an unusable draft — try again");
        return body;
    }

    private static (string System, string User) BuildPrompt(AppFields app, Resume resume)
    {
        var lane = LaneLead.TryGetValue(app.Lane, out var lead) ? lead : LaneLead["ai"];
```

An injectable service that builds an anti-hallucination LLM prompt from structured data and validates output before saving.

```
        var name = resume.FullName.Length > 0 ? resume.FullName : "the candidate";

        var system =
            $"""
            You are an expert cover-letter writer drafting a tailored, ready-to-send cover
            letter for a job applicant. Write in the first person as the applicant.

            Hard rules:
            - Use ONLY the facts in the CANDIDATE BRIEF. Do NOT invent employers, titles,
              metrics, or any claim not present there. Do NOT assert facts about the company
              beyond what is given; you may speak to the role and domain at a general level.
            - Confident, concrete, specific voice. No clichés or filler ("I am excited to",
              "team player", "fast-paced environment", "passionate", "I believe").
            - Lead with the applicant's {lane} strengths and connect them to what THIS role
              and company are about.
            - Plain text / light Markdown only: no preamble or commentary, no code fences,
              no headings, no bullet lists, no placeholders like [Company].

            Structure the letter as:
            - a "Dear Hiring Team," greeting,
            - 2-3 short body paragraphs, ~200-280 words total,
            - a brief sign-off ending with "{name}".
            Do NOT include a date or a postal address.
            """;

        var user =
            $"""
            COMPANY: {Or(app.Company, "(unspecified)")}
            ROLE: {Or(app.Role, "the open role")}
            LOCATION: {Or(app.Location, "(unspecified)")}
            JOB NOTES: {Or(app.Notes, "(none)")}
            POSTING: {Or(app.Link, "(none)")}

            CANDIDATE BRIEF (the only facts you may assert about the applicant):
            {resume.ToBrief()}
            """;

        return (system, user);
    }

    private static string Or(string value, string fallback) => value.Length > 0 ? value : fallback;
}
```

PROJECT

XSpaceWar-AI

A modern, AI-driven networked reimagining of the classic Spacewar!

> A Godot 4 multiplayer space-fighter with deterministic, host-authoritative netcode — clients rebuild the arena from a shared seed and apply snapshots on top. The sample is the wire protocol: a versioned schema guarded by a strict equality check (the only thing standing between mixed builds and silent desync) and authoritative input validation that keeps injection and junk glyphs out of every scoreboard.

GDScript | Godot 4 | Deterministic netcode | Game dev

Host-authoritative wire protocol: a versioned message schema, safe (object-free) encoding, and defensive callsign validation.

```gdscript
class_name NetProtocol
extends RefCounted
## Wire protocol for host-authoritative multiplayer.
##
## Messages are `[type, payload]` arrays encoded with var_to_bytes (and decoded
## with bytes_to_var — never the *_with_objects variants, so a hostile packet
## can't smuggle in scripts/objects). Channel 0 carries reliable control
## traffic (hello/welcome/reject); channel 1 carries the unreliable-sequenced
## state stream (inputs up, snapshots down).
##
## The host runs the only authoritative SimWorld. Clients rebuild the arena
## locally from the seed + params (ArenaGen is deterministic), then apply
## snapshots on top and dead-reckon between them.

# BUMP THIS on ANY wire-schema change (welcome/snapshot/input fields) —
# the strict equality check is the only thing standing between mixed
# builds and silent desync.
const VERSION := 7

## Callsign whitelist: A-Z 0-9 space dash underscore, max 16, never empty.
## Applied authoritatively on the HOST (clients can send anything) — this is
## what keeps BBCode injection, control characters, and unrenderable glyphs
## out of every scoreboard and kill feed.
## Parse "ip" / "ip:port" (one home for the three places that need it —
## the dedicated server's copy had already drifted a validation guard).
static func parse_addr(txt: String, default_port: int) -> Dictionary:
    var t := txt.strip_edges()
    if t == "":
        return {}
    var ip := t
    var port := default_port
    if ":" in t:
        var parts := t.rsplit(":", false, 1)
```

Host-authoritative wire protocol: a versioned message schema, safe (object-free) encoding, and defensive callsign validation.

```
        ip = parts[0]
        if parts.size() > 1 and parts[1].is_valid_int():
            port = int(parts[1])
    return {"ip": ip, "port": port}

static func filter_name(raw: String) -> String:
    var up := raw.strip_edges().to_upper()
    var out := ""
    for ch in up:
        var c := ch.unicode_at(0)
        if (c >= 65 and c <= 90) or (c >= 48 and c <= 57) \
                or ch == " " or ch == "-" or ch == "_":
            out += ch
        if out.length() >= 16:
            break
    return out.strip_edges()

static func sanitize_name(raw: String) -> String:
    var out := filter_name(raw)
    return out if out != "" else "PILOT"

enum {
    MSG_HELLO = 1,      ## client -> host: {v, name, spec?}
    MSG_WELCOME = 2,    ## host -> client: {v, id, seed, prm, mode, dif, gen, ros}
    MSG_REJECT = 3,     ## host -> client: {why}
    MSG_INPUT = 4,      ## client -> host: {q, u, t, f, h, m} (q = input sequence)
    MSG_SNAPSHOT = 5,   ## host -> client: see snapshot_of()
    MSG_PING = 6,       ## client -> host: {t} (sender's ticks_msec, echoed back)
    MSG_PONG = 7,       ## host -> client: {t} (echo)
}

const CH_CONTROL := 0
const CH_STATE := 1
```

PROJECT

embed-clamp

A pure-stdlib proxy that keeps a fixed-context embedder from choking on big RAG chunks.

> A transparent, token-aware proxy for OpenAI-compatible embedding servers: small inputs pass straight through; oversized ones are split to a calibrated token budget, embedded piecewise, and pooled back into one vector — so a fixed-context local embedder never errors on a large document. Zero dependencies. The sample is the splitter and pooling math.

Python | Pure stdlib | RAG / embeddings

Calibrated token-budget splitting (O(pieces) token calls, not O(lines)) and mean/max vector pooling — dependency-free.

```
def ntok(text: str) -> int:
    """Token count for `text`. Exact via the upstream's own tokenizer (POST /tokenize);
    or an estimate (len / chars-per-token) when --approx-chars-per-token is set, for
    servers like TEI/vLLM that don't expose a llama.cpp-style /tokenize."""
    if not text or not text.strip():
        return 0
    cpt = CFG.get("approx_cpt")
    if cpt:
        return math.ceil(len(text) / cpt)
    return len(_post("/tokenize", {"content": text}, CFG["timeout"]).get("tokens", []))


def embed_one(text: str, model: str):
    return _post("/v1/embeddings", {"input": text, "model": model}, CFG["timeout"])["data"][0]["embedding"]


def _char_chunks(s: str, n: int):
    n = max(200, n)
    return [s[i:i + n] for i in range(0, len(s), n)] or [s]


def split_to_budget(text: str):
    """Split text into <=max_tokens pieces, grouping by a calibrated char budget and
    only calling /tokenize to (a) decide if oversized and (b) verify final pieces —
    O(pieces) token calls, not O(lines)."""
    mt = CFG["max_tokens"]
    if len(text) <= mt:                # tokens <= chars for BPE -> definitely fits
        return [text]
    n = ntok(text)
    if n <= mt:
        return [text]
    cpt = max(1.5, len(text) / n)      # chars per token, calibrated from one call
    bud = int(mt * 0.9 * cpt)
    out, cur, clen = [], [], 0
```

Calibrated token-budget splitting (O(pieces) token calls, not O(lines)) and mean/max vector pooling — dependency-free.

```
    for line in text.splitlines(keepends=True):
        if len(line) > bud:            # a single monster line
            if cur:
                out.append("".join(cur)); cur, clen = [], 0
            out.extend(_char_chunks(line, bud))
            continue
        if clen + len(line) > bud and cur:
            out.append("".join(cur)); cur, clen = [line], len(line)
        else:
            cur.append(line); clen += len(line)
    if cur:
        out.append("".join(cur))
    safe = []                          # verify; hard-split the rare straggler
    for p in out:
        if ntok(p) <= mt:
            safe.append(p)
        else:
            safe.extend(_char_chunks(p, int(bud * 0.6)))
    return safe or [text]


def pool(vecs):
    strat = CFG["pool"]
    if strat == "first":
        out = list(vecs[0])
    elif strat == "max":
        out = [max(v[i] for v in vecs) for i in range(len(vecs[0]))]
    else:  # mean
        out = [sum(v[i] for v in vecs) / len(vecs) for i in range(len(vecs[0]))]
    if CFG["normalize"] and strat != "first":
        norm = math.sqrt(sum(x * x for x in out)) or 1.0
        out = [x / norm for x in out]
    return out
```